

Advanced Cryptography

Coursework 2 Discussion

TASK 1

This task presented you with an “extended” dataset: in the corresponding formative assessment, you were asked to perform several experiments with different distinguisher configuration, as well as SNR analyses, on a dataset called WS1. The dataset WS2, which underpins this task, was sampled from the same device, but this time over an extended period of time. Hence, WS2 enables to recover the entire roundkey of an (unprotected) AES implementation.

As a consequence, the correlation, DoM, and SNR scripts that I supplied you with for the formative assignment(s) work “out of the box”; you just need to give them the appropriate input data.

Assuming that you used the code fragment that I provided to extract the actual data from the .h5 file; the task then boiled down to writing a for loop that iterates over the 16 statebytes/keybytes for an AES round and evaluates, e.g. the PCor distinguisher (suitably configured, as per the instructions of the assignment).

TASK 1, CONT.

```
for keybyte in range(16):
    pcor_vals = PCor(inputs[:, keybyte], traces)
    key_ind = np.unravel_index(np.argmax(np.absolute(pcor_vals)),
                              pcor_vals.shape)
    print(f"[+] Highest correlation occurs for key {key_ind[0]} at time
          index {key_ind[1]}.")
```

Listing: Code fragment to do Correlation on a keybyte

I did not specify a power model subtasks 1 and 2, so I will accept any choice. I asked that you give me a list of top ranked keys (the code fragment extracts only the top ranked key): any length list is good. The most natural interpretation of “top ranked” is that you identify the point in time (ct) for (ck) and then print a list of the best x keys at time point ct . But if you elected to select x time points and plot the keys for these, then this is good too.

TASK 1, CONT.

Subtask 3 is more specific: it asks you explicitly to assume a HW adversary, which means that you need to select as label `hw_sb` in the `doSNR` function, and you needed to ensure that you set the key to ck_i (with i referring to the i -th key byte) when computing `ark`. But the SNR functionality that I gave you had, in principle, all the code that you needed.

Plotting things into the same plot should have been made easier as made code examples demonstrated how to plot several plots into the same figure, using some colour control, etc.

TASK 2

Task 2 was about you showing that the table pre-computation step that is commonly done (as a time-memory trade-off) prior to a masked encryption, actually adds another nice target for a differential input attack.

The idea is that in the computation of $S_m(x + m) = S(x) + m'$, one knows the 256 values of x , and therefore can use them as differential inputs in order to extract the one-time random values m, m' . The four subtasks now guide you through showing this concretely.

The first subtask required you to write a simulation for the leakage that one can expect to occur during the computation of $S_m()$. I gave you code that shows how to simulate something else in relation to a masked implementation. From this code you could see how to simulate noisy HW leakage, e.g. the leakage on i can be simulated via:

```
traces[i,0] = HW[i] + rng.normal(0, 0.1)
```

(the simulated leakage is stored in a numpy array called `traces` of size (256 x 6)).

TASK 2, CONT.

Doing this simulation for all six intermediate values (I required that you simulate leakage for $i, m, m', i \oplus m, S(i \oplus m), S(i \oplus m) \oplus m'$), with S being SubBytes, then gives the array `traces`, which has $i = 256$ rows and 6 columns.

The differential inputs are the values $0, \dots, 255$, let's say you store them in a variable called `inputs` (as a column vector).

The you can call `PCor`, configured to do HW on SubBytes, to extract m from the simulated leakage $S(i \oplus m)$:

```
pcor_vals = PCor(inputs, traces).
```

TASK 2, CONT.

To extract m' you need to target $S(x \oplus m) \oplus m'$. One way to do this is to first extract m , then using knowledge of m you pass in as inputs $S(x \oplus m)$ and configure PCor to do an attack using `ark`. Another option would be to write a custom correlation function and guess m, m' simultaneously.

You should find that, in line with what you have found in the formative exercises, the result of SubBytes is easier to exploit than the result of AddRoundKey, showing that the target function (aka which intermediate value we go for) has an impact on attack performance. I now characterise attack performance, in line with the lecture notes, via the number of needed power traces.

You would also find that increasing the standard deviation s , means that we have more noise on the simulated measurements, and therefore we need more traces. Sadly, we only have 256 traces available. Now, depending on how exactly you define “attack success”, you will report different SNRs as “lowest SNR” that enables recovery of m, m' .

TASK 3

Task 3 has three subtasks.

The first one is book work: it required you to check the lecture notes/slides to find the part where I explained that SubBytes is in fact a function that requires a finite field inversion. When using a $(2, 2)$ Boolean secret sharing scheme, we now have to consider how to deal with the inversion as sadly $S(a \oplus m) \neq S(a) \oplus m$. The provided function takes in $a \oplus m$, then switches the Boolean masking to multiplicative masking $a \cdot y$, so that the inverse can be done “easily” giving $a^{-1} \cdot y^{-1}$, and then switching back to Boolean masking.

The second subtask asked you to look at the provided function and consider if this is more memory efficient than what we do in the classical Boolean $(2, 2)$ secret sharing (aka Task 2). The classical option requires to store S and S_m , so 512 bytes in total.

TASK 3, CONT.

The function `masked_inverse()` seems to not require any table. But this is only because the pseudo-code hides somewhat that the multiplication and the inversion are done over a finite field. Now there are multiple possible answers:

- ▶ perhaps a device natively supports this kind of finite field arithmetic. In this case no extra memory is required.
- ▶ if these operations are not available then there are two options. First, one has dedicated functions that do multiplication and inversion without any look-up tables: thus no extra memory. Second, there are dedicated functions with tables, or tables. Thus there is more memory required.

Any answer among the options is fine. A brilliant answer would recognise that this is a “it depends” situation.

TASK 3, CONT.

The third subtask then asks you to think about the security of the code and argue security/insecurity both theoretically and practically.

You know that most of the time I give you something that is insecure, and your job is to find an example that showcases this.

Here, I pointed several times during the lectures to the fact that a multiplication is zero whenever one of the operands is zero. This is why multiplicative masking is a bad idea: in the code you have that v_5 leaks $a \cdot y$. Even if you choose the multiplicative mask y to be non-zero, it can be that $a = p \oplus k$ can be zero (this is whenever $p = k$).

TASK 3, CONT.

(a): The theoretical argument would be to show that v_5 (and also v_6) require knowledge of a to write a simulator for the Random world. You can do this by simply writing out what each step in algorithm `masked_inverse()` represents.

(b): There are (at least) two options to provide a practical demonstration. If you write a simulation for the code that saves the HW of the intermediate steps into traces, aka by doing something like `leak[i, 4] = HW[v5]`, then you can show that whenever $a = 0$, certain intermediate leaks are zero (and when $a \neq 0$ then they are non-zero). This is really easy to do, and I accept this as an answer (even if you don't fully implement the proper multiplication/inversion). It is possible, using correlation as a distinguisher to turn this into a key recovery attack.

TASK 3, CONT.

To do the “full attack” you should use the Galois package to do the finite field arithmetic: I use the Galois package to just compute tables for the inversion and multiplication:

```
def gen_FF_tables():
    GF = galois.GF(2**8)
    inv = np.zeros((256,1))
    mult = np.zeros((256,256))
    for i in range(256):
        if i==0:
            inv[i] = 0
        else:
            inv[i] = GF(i)** -1
    for a in range(256):
        for b in range(256):
            mult[a,b] = GF(a) * GF(b)
    return inv, mult
```

TASK 3, CONT.

Then, using the tables `inv[i]`, `mult[a,b]` one can write a simulation looping over n input bytes that produces simulated leaks like

```
inputs = rng.integers(low=0, high=256, size=n)
for i in range(n):
    # generate input for function masked_inverse

    a = np.bitwise_xor(inputs[i], ck)
    x = rng.integers(low=0, high=256, size=1)
    y = rng.integers(low=1, high=256, size=1)
    [SNIP]
    v5 = np.bitwise_xor(v4,v2)
    leak[i,4] = int(HW[v5])
    v6 = int(inv[v5])
    leak[i,5] = int(HW[v6])
    [SNIP]
```

TASK 3, CONT.

The attack then proceeds by using a power model that assigns 0 as predicted leakage whenever the intermediate a is predicted to be zero, and something else otherwise. In my code snippet below I assign the value -1 as predicted values for all $a > 0$.

Like in any differential input attack, predictions are made for all 256 values that a key byte can take. Below is a code snippet for illustration. As the zero-value power model is not included in PCor, it is better to just call `np.corrcoef()` directly.

```
pred_a = np.bitwise_xor(inputs, k)
leak_a = np.where(pred_a > 0, -1, pred_a)

corr = np.corrcoef(pred_a, leak[:,5], rowvar=False)
```

Listing: One incorrect key guess vs the correct key guess

With this configuration you should find that the correct key will give the highest (negative) correlation (around -0.17 using enough traces).